

Theory of Computation

Lecture 5: Nondeterminism and Nondeterministic Finite Automata

Sheikh Azizul Hakim

Lecturer, Department of CSE, BUET

Last compiled: June 27, 2026 6:05pm Z

Sometimes finding a solution is hard.
But verifying a guessed solution is easy.

Lecture 5A: The Philosophy of Nondeterminism

The Central Question

In deterministic computation, every step is forced.

(current state, input symbol) $\longrightarrow$ exactly one next state

Today we ask:

What if a machine could explore many possibilities?

The Central Question

In deterministic computation, every step is forced.

(current state, input symbol) $\longrightarrow$ exactly one next state

Today we ask:

What if a machine could explore many possibilities?

This idea is called **nondeterminism**.

Find versus Verify

Many computational tasks have two versions.

Problem	Find	Verify
Maze	Find a path	Check a given path
Sudoku	Solve puzzle	Check a completed board
Password	Discover password	Test a candidate password
SAT	Find assignment	Check a given assignment
Proof	Find proof	Check a written proof

Find versus Verify

Many computational tasks have two versions.

Problem	Find	Verify
Maze	Find a path	Check a given path
Sudoku	Solve puzzle	Check a completed board
Password	Discover password	Test a candidate password
SAT	Find assignment	Check a given assignment
Proof	Find proof	Check a written proof

The second task is often much easier.

Example: Maze

Task:

Find a path from the entrance to the exit.

This may require search.

Example: Maze

Task:

Find a path from the entrance to the exit.

This may require search.

But if someone gives us a candidate path, we can quickly check:

- Does it start at the entrance?
- Does every step follow a valid corridor?
- Does it end at the exit?

Example: Maze

Task:

Find a path from the entrance to the exit.

This may require search.

But if someone gives us a candidate path, we can quickly check:

- Does it start at the entrance?
- Does every step follow a valid corridor?
- Does it end at the exit?

Finding and verifying are different kinds of computation.

Example: Sudoku

Finding a solution may be difficult.

Verifying a completed board is straightforward:

- Check every row.
- Check every column.
- Check every block.

Example: Sudoku

Finding a solution may be difficult.

Verifying a completed board is straightforward:

- Check every row.
- Check every column.
- Check every block.

This distinction will reappear later in complexity theory as a central theme.

The Nondeterministic Dream

Imagine a machine that can say:

Let me guess the correct branch of the search.

If the guess can be verified, the machine accepts.

The Nondeterministic Dream

Imagine a machine that can say:

Let me guess the correct branch of the search.

If the guess can be verified, the machine accepts.

This is not how ordinary computers physically work.

But it is an extremely useful mathematical abstraction.

Three Ways to Think About Nondeterminism

1. **Magical guessing**

The machine guesses the correct choice.

2. **Parallel universes**

All choices are explored simultaneously.

3. **Computation tree**

The computation branches into many paths.

Three Ways to Think About Nondeterminism

1. **Magical guessing**

The machine guesses the correct choice.

2. **Parallel universes**

All choices are explored simultaneously.

3. **Computation tree**

The computation branches into many paths.

For finite automata, these viewpoints describe the same idea.

Does Nondeterminism Exist Physically?

Probably not as a literal machine that always guesses correctly.

But theoretical computer science often studies idealized models.

Examples:

- infinite tapes,
- real numbers,
- perfect randomness,
- nondeterministic choices.

Does Nondeterminism Exist Physically?

Probably not as a literal machine that always guesses correctly.

But theoretical computer science often studies idealized models.

Examples:

- infinite tapes,
- real numbers,
- perfect randomness,
- nondeterministic choices.

Idealized models help us understand the structure and limits of computation.

Why Study It Then?

Nondeterminism models the shape of search.

It appears behind many real computational tools:

- backtracking algorithms,
- SAT solvers,
- theorem provers,
- symbolic execution,
- program analysis,
- AI planning,
- regex engines.

**Nondeterminism is not merely an automata trick.
It is one of the central ideas of theoretical computer science.**

Lecture 5B: From DFA to NFA

Recall: DFA

A deterministic finite automaton has

$$M = (Q, \Sigma, \delta, q_0, F).$$

The transition function is

$$\delta : Q \times \Sigma \rightarrow Q.$$

For each state and input symbol, there is exactly one next state.

What Changes in an NFA?

In a nondeterministic finite automaton, one situation may have many possible next states.

Instead of

$$(q, a) \mapsto q'$$

we allow

$$(q, a) \mapsto \{q_1, q_2, \dots, q_k\}.$$

The machine may branch.

Let

$$\text{CONTAINS}_{101} = \{w \in \{0,1\}^* : w \text{ contains } 101 \text{ as a substring}\}.$$

Examples in the language:

101, 0101, 1010, 1110100.

Examples not in the language:

ε , 0, 11, 1000, 0100.

To recognize whether the input contains 101, a DFA keeps track of partial progress.

It remembers whether it has recently seen:

nothing useful, 1, 10, 101.

This is perfectly possible.

To recognize whether the input contains 101, a DFA keeps track of partial progress.

It remembers whether it has recently seen:

nothing useful, 1, 10, 101.

This is perfectly possible.

But there is another way to think.

Whenever the machine sees a 1, it may guess:

This 1 is the beginning of 101.

If the guess is correct, one branch reaches an accepting state.

If the guess is wrong, that branch dies.

Whenever the machine sees a 1, it may guess:

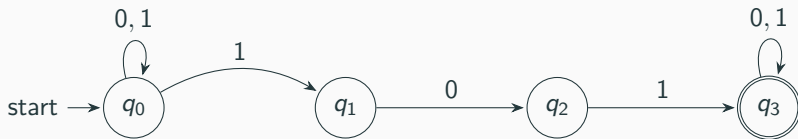
This 1 is the beginning of 101.

If the guess is correct, one branch reaches an accepting state.

If the guess is wrong, that branch dies.

The input is accepted if at least one guess succeeds.

NFA for Strings Containing 101



The branch $q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_3$ verifies one guessed occurrence of 101.

Why Is This Nondeterministic?

At state q_0 , on input 1, the machine has two choices:

$$\delta(q_0, 1) = \{q_0, q_1\}.$$

Interpretation:

- stay in q_0 : ignore this 1;
- move to q_1 : guess this 1 starts the pattern.

Both possibilities are explored.

Trace on Input 0101

Input:

0101

At the final 1, there is a branch that has seen

$1 \rightarrow 10 \rightarrow 101.$

So one branch reaches q_3 .

$q_3 \in F$

Therefore, 0101 is accepted.

Trace on Input 1000

Input:

1000

The machine may guess the first 1 begins 101.

Trace on Input 1000

Input:

1000

The machine may guess the first 1 begins 101.

Then it reads 0, which is promising.

Trace on Input 1000

Input:

1000

The machine may guess the first 1 begins 101.

Then it reads 0, which is promising.

But the next symbol is 0, not 1.

That branch dies.

Trace on Input 1000

Input:

1000

The machine may guess the first 1 begins 101.

Then it reads 0, which is promising.

But the next symbol is 0, not 1.

That branch dies.

No accepting branch exists.

Therefore, 1000 is rejected.

Important rule:

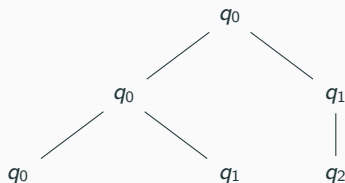
An NFA accepts if at least one computation path accepts.

Consequences:

- If one path accepts and many paths reject, the input is accepted.
- If all paths reject or die, the input is rejected.

Computation Tree View

For an NFA, computation is a tree.



If any leaf is accepting after the input is consumed, the NFA accepts.

A DFA has one timeline.

An NFA has many possible timelines.

Lecture 5C: Formal Definition of NFA

Formal Definition

A nondeterministic finite automaton is a 5-tuple

$$M = (Q, \Sigma, \delta, q_0, F),$$

where:

- Q is a finite set of states,
- Σ is the input alphabet,
- $q_0 \in Q$ is the start state,
- $F \subseteq Q$ is the set of accepting states,
- δ is the transition function.

Only δ differs from the DFA case.

DFA versus NFA Transition Functions

For a DFA:

$$\delta : Q \times \Sigma \rightarrow Q.$$

For an NFA:

$$\delta : Q \times \Sigma \rightarrow 2^Q.$$

Here 2^Q means the set of all subsets of Q .

Why a Set of States?

Suppose

$$Q = \{q_0, q_1, q_2\}.$$

Then

$$2^Q = \{\emptyset, \{q_0\}, \{q_1\}, \{q_2\}, \{q_0, q_1\}, \{q_0, q_2\}, \{q_1, q_2\}, \{q_0, q_1, q_2\}\}.$$

An NFA transition may return any one of these subsets.

Interpreting the Output of δ

$$\delta(q, a) = \emptyset$$

means no branch survives.

$$\delta(q, a) = \{r\}$$

means exactly one next possibility.

$$\delta(q, a) = \{r, s, t\}$$

means the computation branches into three possibilities.

Formal Specification of the 101 NFA

$$Q = \{q_0, q_1, q_2, q_3\}, \quad \Sigma = \{0, 1\}, \quad F = \{q_3\}.$$

q_0 is the start state.

The transition function is:

δ	0	1
q_0	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\{q_2\}$	$\emptyset$
q_2	$\emptyset$	$\{q_3\}$
q_3	$\{q_3\}$	$\{q_3\}$

Notice the Difference

In a DFA table, each entry is a state.

$q_0, \quad q_1, \quad q_2$

In an NFA table, each entry is a set of states.

$\emptyset, \quad \{q_0\}, \quad \{q_0, q_1\}$

The table is just a precise way to define the transition function.

Every DFA Is an NFA

A DFA transition

$$\delta(q, a) = r$$

can be viewed as the NFA transition

$$\delta(q, a) = \{r\}.$$

So every DFA is automatically an NFA.

Every DFA Is an NFA

A DFA transition

$$\delta(q, a) = r$$

can be viewed as the NFA transition

$$\delta(q, a) = \{r\}.$$

So every DFA is automatically an NFA.

But is every NFA equivalent to some DFA?

That is the big question.

Lecture 5D: More Examples

Example: Strings Ending with 01

Let

$$L = \{w \in \{0,1\}^* : w \text{ ends with } 01\}.$$

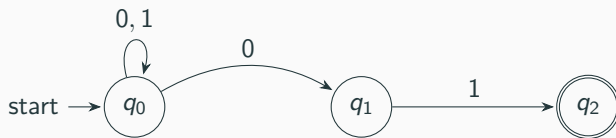
Examples:

01, 101, 0001, 1101.

Non-examples:

ε , 0, 1, 010, 011.

NFA for Strings Ending with 01



The machine guesses which 0 is the second-last symbol.

Why This Works

For input

110101

the NFA may ignore early symbols.

When it sees the final 0 before the last 1, it guesses:

This 0 is the beginning of the suffix 01.

If the next symbol is 1 and input ends, the machine accepts.

Important Subtlety

In the NFA for strings ending with 01, the accepting state has no outgoing transition.

Why?

In the NFA for strings ending with 01, the accepting state has no outgoing transition.

Why?

If the machine reaches the accepting state before the input ends, that branch dies on the next input symbol.

Thus acceptance requires ending exactly at the accepting state.

Example: Third Symbol from the End Is 1

Let

$$L = \{w \in \{0,1\}^* : \text{the third symbol from the end of } w \text{ is } 1\}.$$

Examples:

100, 101, 1101, 00100.

Non-examples:

ε , 0, 10, 000, 0100.

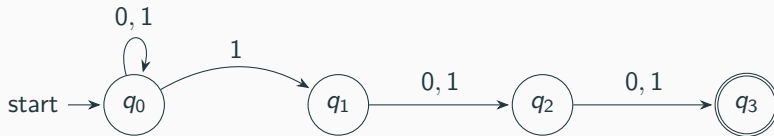
The machine scans the input.

Whenever it sees a 1, it may guess:

This 1 is the third symbol from the end.

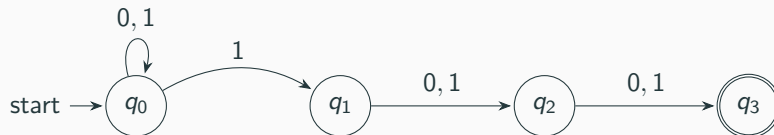
Then it must read exactly two more symbols and stop.

NFA for Third Symbol from the End Is 1



This NFA has only four states.

NFA for Third Symbol from the End Is 1



This NFA has only four states.

The equivalent DFA may need to remember the last three bits.

Why NFAs Can Be Shorter

NFAs can express:

“There exists a good guess.”

For the language

third symbol from the end is 1,

the good guess is the position of that special 1.

DFAs cannot guess; they must remember enough information deterministically.

Example: At Least One 0 and At Least One 1

Let

$$L = \{w \in \{0,1\}^* : w \text{ contains at least one 0 and at least one 1}\}.$$

Examples:

01, 10, 001, 1110.

Non-examples:

ϵ , 000, 111.

A DFA may remember which symbols have appeared so far.

none, only 0, only 1, both.

This gives four natural states.

A DFA may remember which symbols have appeared so far.

none, only 0, only 1, both.

This gives four natural states.

Here nondeterminism is not especially useful.

Not every language becomes simpler with NFAs.

Example: Exactly the String 101

Let

$$L = \{101\}.$$

This language contains exactly one string.

Example: Exactly the String 101

Let

$$L = \{101\}.$$

This language contains exactly one string.

A machine for this language should accept 101, and reject everything else:

ϵ , 1, 10, 1010, 0101.

NFA for $\{101\}$



No outgoing transitions are needed.

A missing transition means that branch dies.

Lecture 5E: Special Languages and Empty Objects

A Common Confusion

Do not confuse:

$\emptyset$

and

$\{\epsilon\}$.

They are very different languages.

The Empty Language

$$\emptyset$$

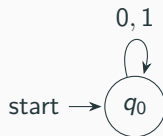
contains no strings.

Not even the empty string.

So:

$$\epsilon \notin \emptyset.$$

NFA for the Empty Language



There are no accepting states:

$$F = \emptyset.$$

Therefore, no input is accepted.

The Language Containing Only the Empty String

$$\{\varepsilon\}$$

contains exactly one string: the empty string.

So:

$$\varepsilon \in \{\varepsilon\}.$$

But:

$$0 \notin \{\varepsilon\}, \quad 1 \notin \{\varepsilon\}.$$

NFA for $\{\varepsilon\}$



The start state is accepting.

But there are no transitions on 0 or 1, so any nonempty input is rejected.

Compare Carefully

$\emptyset$

has zero strings.

$\{\varepsilon\}$

has one string.

Compare Carefully

$\emptyset$

has zero strings.

$\{\varepsilon\}$

has one string.

Therefore:

$\emptyset \neq \{\varepsilon\}.$

More Special Languages

Over alphabet $\Sigma = \{0, 1\}$:

$$\Sigma^*$$

all binary strings.

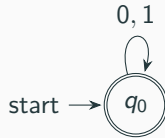
$$\Sigma^+$$

all nonempty binary strings.

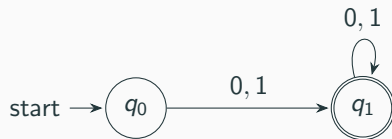
$$\emptyset$$

no strings.

$$\{\varepsilon\}$$



Every string is accepted, including ϵ .



All nonempty strings are accepted.

The empty string is rejected.

Finite Languages Are Regular

Every finite language can be recognized by a finite automaton.

Example:

$$L = \{00, 11, 101\}.$$

We can build one path for each string.

Finite Languages Are Regular

Every finite language can be recognized by a finite automaton.

Example:

$$L = \{00, 11, 101\}.$$

We can build one path for each string.

Nondeterminism makes this construction very natural.

For

$$L = \{00, 11, 101\},$$

create a new start state and branch into three paths:

00, 11, 101.

This idea will later become useful for regular expressions and closure properties.

Lecture 5F: Epsilon Transitions

Can the Machine Move Without Reading Input?

An NFA may contain transitions labeled

ϵ .

These consume no input symbol.

They are called ϵ -**transitions**.

How to Think About ε -Transitions

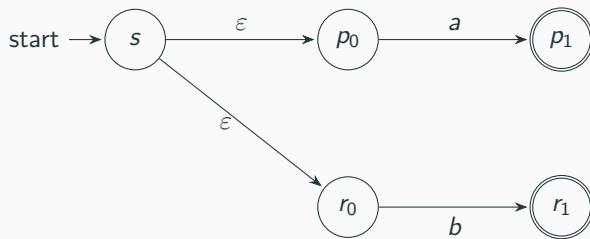
An ε -transition means:

The machine changes state without reading input.

Possible interpretations:

- changing its mind,
- jumping to a submachine,
- making a free nondeterministic choice,
- starting a different module.

Example: Accepting $\{a, b\}$



The machine freely chooses the a -path or the b -path.

Why ϵ -Transitions Are Useful

They make automata modular.

If we already have machines for L_1 and L_2 , ϵ -transitions let us combine them easily.

This will later help us prove closure under:

- union,
- concatenation,
- star.

Union Idea

To recognize

$$L_1 \cup L_2,$$

make a new start state.

Use ϵ -transitions to enter either machine:

$$\text{new start} \xrightarrow{\epsilon} M_1 \quad \text{or} \quad \text{new start} \xrightarrow{\epsilon} M_2.$$

Accept if either submachine accepts.

Concatenation Idea

To recognize

$$L_1L_2,$$

run the machine for L_1 , then jump to the machine for L_2 .

Use ε -transitions from accepting states of M_1 to the start state of M_2 .

The NFA guesses where the first part ends and the second part begins.

Star Idea

To recognize

$$L^*,$$

we must allow:

$$0, 1, 2, 3, \dots$$

copies of strings from L .

ϵ -transitions allow us to loop back and repeat the machine.

This is one reason NFAs are so convenient.

Lecture 5G: Nondeterminism in the Larger Story

Nondeterminism and Regular Languages

NFAs do not change which languages are regular.

But they often make descriptions simpler.

They are useful because they separate two questions:

- How naturally can we describe a language?
- How powerful is the model?

Nondeterminism Beyond Automata

The same idea reappears in later topics:

- nondeterministic Turing machines,
- the class NP,
- SAT and NP-completeness,
- search problems,
- proof verification,
- optimization,
- automated reasoning.

This is why NFA is more than a technical definition.

Later in complexity theory, you will see:

NP

as the class of problems where a proposed solution can be verified efficiently.

Nondeterminism gives one mathematical way to model guessing such a solution.

NFA is our first small encounter with that idea.

Think Before Next Lecture

We now have two models:

DFA and NFA.

NFAs seem more flexible.

They can branch.

They can guess.

They can use ε -transitions.

Question:

Do they recognize more languages?

A nondeterministic machine seems more powerful.

Next lecture:

Every NFA can be simulated by a DFA.

Practice Problems

Practice: Predict the Language

For each NFA, determine the language.

1. A single accepting start state with no transitions.
2. A single non-accepting start state with loops on 0, 1.
3. A path labeled 0, 1, 0 ending in an accepting state.
4. A start state with ε -branches to paths 00, 11, and 101.

Practice: Design NFAs

Design NFAs for:

1. strings ending with 101;
2. strings containing 000;
3. strings whose fourth symbol from the end is 1;
4. $\{0, 11, 101\}$;
5. all nonempty strings over $\{a, b\}$.

Key Takeaways

- A DFA has exactly one next state.
- An NFA may have many possible next states.
- A branch may die.
- An input is accepted if at least one branch accepts.
- ε -transitions consume no input.
- NFAs often make language descriptions simpler.
- Nondeterminism is a central abstraction in TCS.

Nondeterminism is the mathematics of possibility.

Next:

possibility can still be simulated deterministically.